

Prompt Engineering Coding Challenges

Below are **22 coding/debugging/problem-solving challenges**, each aligned with a different prompt engineering technique. These are designed for students to practice applying diverse prompting strategies to real coding problems.

1. Role Prompting – Code Reviewer

You are a Python code reviewer. Review and improve this code for readability and efficiency:

```
numbers = [1,2,3,4,5,6,7,8,9,10]
squares = []
for n in numbers:
    if n % 2 == 0:
        squares.append(n*n)
print(squares)
```

2. Chain of Thought – Algorithm Design

Step by step, design a function that returns the first non-repeating character in a string.
Example: "stress" → "t".

3. ReAct – Debugging with Reasoning

This function is supposed to calculate the average, but fails in some cases:

```
def average(nums):
    return sum(nums) / len(nums)
print(average([]))
```

First explain why it breaks, then propose a safe fix.

4. Few-Shot Prompting – Pattern Learning

Examples:

- Input: 3, 4 → Output: 12

- Input: 5, 6 → Output: 30

New task: design a function that follows this pattern.

5. Zero-Shot Prompting – Data Cleaning

Fix this buggy code in one shot:

```
data = ["apple", "banana", None, "orange", "", "kiwi"]
cleaned = [x.strip().lower() for x in data]
print(cleaned)
```

6. Self-Consistency – Multiple Solutions

Write two different approaches to **reverse a string** in Python. Then explain which one is simpler for beginners to understand.

7. ReAct – Logic Bug

This code should print only prime numbers from 1–20, but it doesn't:

```
for num in range(1, 21):
    for i in range(2, num):
        if num % i == 0:
            print(num, "is prime")
```

8. Role Prompting – Mentor Explainer

Pretend you're mentoring a beginner. Explain why the following recursion fails and how to fix it:

```
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n)
print(factorial(5))
```

9. Prompt with Constraints

Fix this code **without using built-in functions like max() or sorted()**:

```
nums = [3, 7, 2, 9, 5]
print(max(nums))
```

10. Comparative Prompting – Trade-off Analysis

Write two solutions for removing duplicates from a list while keeping order. Which one would scale better for very large lists?

11. Reflective Prompting – Beginner Mistake

Why does this code behave unexpectedly? Suggest a corrected version.

```
x = [1, 2, 3]
y = x
y.append(4)
print(x)
```

12. Creative Prompting – Design + Debug

Invent a small function that calculates something interesting (e.g., Fibonacci numbers, word frequency counter), then **intentionally insert a bug** and write prompts to help debug it.

13. Socratic Prompting – Guided Debugging

Guide yourself through questions to fix this code:

```
for i in range(5):
    if i = 3:
        print("Found 3")
```

14. Zero-Shot Chain of Thought – Edge Case Handling

Write a function to divide two numbers. Handle division by zero gracefully in your reasoning and code.

15. Analogical Prompting – By Example

Given that `len([1,2,3])` works for lists, design a function `count_chars()` that behaves similarly for strings.

16. Meta Prompting – Self-Instruction

“Explain how you will solve the problem first, then provide the code.”

Task: Generate all even numbers up to 50.

17. Multi-Persona Prompting – Team of Experts

Act as a team: one persona focuses on **efficiency**, the other on **readability**. Both propose solutions to summing digits of a number. Then agree on the best one.

18. Chain-of-Verification – Double Check

Write a function to check if a word is an anagram of another. Then verify your answer with a different approach.

19. Reflective Self-Correction

Intentionally provide a wrong solution to sorting a list, then reflect and correct it.

20. Step-by-Step Translation Prompt

Convert this Python function into JavaScript step by step:

```
def greet(name):  
    return f"Hello, {name}!"
```

21. Error Prediction Prompt

Predict what error this code will raise before running it, then fix it:

```
print(10/0)
```

22. Optimization Prompt

This code works but is inefficient. Optimize it:

```
result = []  
for i in range(1, 10001):
```

```
if i % 2 == 0:  
    result.append(i)
```